Workload-Shaped Serving Optimization for SmolLM2-360M on CPU and RTX 6000 Ada

Hanxiao Haiyang Peng
DSAA4012 Final Project

***Abstract*—We characterize how batch size and context length reshape the time-to-first-token (TTFT), time-per-output-token (TPOT), throughput, and memory frontier of SmolLM2-360M-Instruct. A deterministic manual decode loop separates prefill from steady decode on one AMD EPYC 9654 socket and one NVIDIA RTX 6000 Ada GPU. We then evaluate scaled-dot-product attention (SDPA), key-value (KV) caching, and dynamic W8A8 quantization. The result is conditional rather than universal. SDPA becomes increasingly valuable at long context. Cache bookkeeping can slightly hurt the smallest GPU loads, but avoiding the cache becomes 4.22× slower at context 2048, batch 8. CPU W8A8 improves throughput by 1.27–2.49×, yet increases peak resident memory and severely degrades perplexity and ARC-Easy accuracy. Correcting the benchmark to retain only final-position logits also removes artificial out-of-memory boundaries, demonstrating that measurement semantics are part of the systems result.**

***Index Terms*—language-model serving, TTFT, TPOT, batching, attention, KV cache, quantization**

I. Problem and System Overview

Serving optimization is often reduced to one tokens-per-second number. That summary hides a multi-objective problem: a serving path must balance prefill latency, decode latency, aggregate throughput, memory capacity, and output quality. Our primary question is therefore a frontier question: *how do batch size and context length reshape the TTFT–TPOT–throughput frontier, and when do attention, caching, and quantization move it?*

The model is `HuggingFaceTB/SmolLM2-360M-Instruct`, a compact decoder-only Llama-family model from the SmolLM2 family [1]. The checked artifact contains 361,821,120 parameters, 32 decoder layers, hidden size 960, intermediate size 2560, 15 query heads, 5 KV heads, grouped-query attention, and an 8192-token maximum context. The small model is deliberate: at small batch it is unlikely to saturate a high-end GPU, making framework overhead, dispatch, bandwidth, attention, and matrix-throughput transitions visible within a tractable experiment budget.

The study has two independent hardware lines. The CPU line uses logical CPUs 0–95 from one AMD EPYC 9654 NUMA/socket side, with 96 intra-op threads, one inter-op thread, and FP32 weights. The GPU line uses only GPU0 of an eight-GPU host: one RTX 6000 Ada with 48 GB memory and BF16 weights. Absolute CPU and GPU results are not treated as a fairness comparison; we compare scaling behavior and normalized intervention effects within each line.

II. Design

A. Timing Boundaries and Metrics

The benchmark separates autoregressive inference into prefill and decode. Prefill consumes a prepared prompt and ends when the first generated token is available. Decode then performs one forward step per sequence and generates 64 output tokens synchronously with greedy selection. Input construction, tokenization, and terminal output are outside the measured region. CUDA is synchronized at every token boundary.

For a generation producing N tokens, model-only TTFT is the elapsed time from the first model call to availability of the first generated token. Steady decode latency is

$$\mathrm{TPOT} = \frac{t_{\mathrm{last}} - t_{\mathrm{first}}}{N-1}, \tag{1}$$

and aggregate output-token throughput is

$$\mathrm{TPS} = \frac{BN}{t_{\mathrm{last}} - t_{\mathrm{start}}}, \tag{2}$$

where B is batch size. Prompt tokens are excluded from decode TPS. GPU peak allocated and reserved memory and fresh-process CPU peak RSS are recorded.

B. Workload Grid and Statistical Protocol

Baseline grids span contexts $\{128, 512, 2048, 4096\}$, CPU batches $\{1, 2, 4, 8, 16\}$, and GPU batches $\{1, 2, 4, 8, 16, 32, 64, 128\}$. Each broad-grid point has ten measured repetitions after warmup. Two GPU headline points, 128/1 and 4096/128, have 100 repetitions collected in ten alternating-order blocks. We report median, p95, and p99, but use the 100-repetition files for stable tail claims.

The batching knee is frozen before analysis: it is the first tested batch increase for which aggregate TPS improves by less than 10% while p95 TTFT or TPOT continues to rise. If no tested point meets the rule, we state that no knee was observed rather than extrapolating one.

C. Interventions and Quality

We compare PyTorch eager attention with SDPA. PyTorch SDPA can dispatch to fused backends [2]; CUDA profiling verifies that our selected path executes `pytorch_flash::flash_fwd_kernel`, consistent with the IO-aware design principles of FlashAttention [3]. Eager profiles instead expose explicit batched matrix multiplies and softmax.

KV cache enabled and disabled are compared at representative context/batch points. Dynamic W8A8 quantization is evaluated on CPU with 224 Transformer linear modules quantized and `lm_head` retained in FP32. Quality is measured with a frozen 12,846-token WikiText-2 slice [4], full ARC-Easy [5], and full HellaSwag [6].

III. Implementation

A. Reproducible Harness

The repository provides YAML-composed configurations, deterministic prompt construction, a manual prefill/decode loop, token-level timing, memory measurement, JSONL schema validation, resume-safe manifests, aggregation, plotting, and quality-evaluation scripts. Each result records the merged configuration, exact software versions, local model hashes, Git revision, and invocation history. The final repository passes 22 model-free unit tests and Ruff static checks.

B. Correctness Boundary: Final-Position Logits

An early implementation materialized full-sequence logits with shape $[B, L, V]$ during prefill, even though greedy serving uses only the last position. The corrected schema-v3 benchmark requests `logits_to_keep=1`, producing $[B, 1, V]$ on every forward. Legacy records remain explicitly labeled `full_logits_prefill` and are never combined with corrected aggregates.

This correction materially changes the measured memory boundary. Former OOM points at 2048/128 and 4096/64 succeed in fresh processes and each uses about 16.33 GiB peak allocated memory. The 4096/128 point also succeeds at 31.95 GiB. Residual memory beyond weights and KV state scales with $B \times L$, consistent with transient prefill QKV and MLP activations rather than unused vocabulary logits. The correction is therefore not merely an implementation detail; it changes which workloads appear feasible.

C. Validation and Interpretation Discipline

The corrected baseline contains 200 CPU and 320 GPU observations. Operator experiments cover five representative points per hardware line with ten repetitions per implementation. Cache experiments use repeated points except for one explicitly labeled CPU boundary. Hardware, environment, test, lint, telemetry, profiler, raw, processed, and plot artifacts are committed together.

A faster quantized result is not attributed to INT8 without checking the executed kernel. A slower quantized path is retained and analyzed. Memory claims distinguish GPU allocator statistics from fresh-process CPU RSS. Broad-grid p99 values based on ten repeats are descriptive only. Block telemetry spans 29–70°C without progressive failure or a memory leak.

IV. Evaluation

A. Baseline Surface

Fig. 1 shows the corrected median decode-TPS surfaces. CPU median TPS peaks at 146.96, 119.91, 50.95, and 25.01 for contexts 128, 512, 2048, and 4096, respectively. The knee rule finds batch 16 at context 2048 and batch 8 at context 4096; no knee appears through batch 16 at contexts 128 or 512.

GPU median TPS reaches 2973.59, 2947.87, 2700.83, and 1437.52 at batch 128 for the same four contexts. Only context 4096 reaches the frozen knee at batch 128. The other contexts still scale within the tested range. The 100-repetition 128/1 point has p50/p95/p99 TTFT of 41.54/44.08/44.67 ms. At 4096/128 these values are 6.745/6.899/6.923 s; p99 TPOT is 89.48 ms.

TABLE I
Normalized Intervention Effects

Study	Measured effect	Interpretation
SDPA	4096/4 TTFT: 6.23× CPU, 9.34× GPU	Clear long-context win; fused CUDA path verified.
KV cache	GPU off/on TPOT: 0.95–0.99× at small loads; 4.22× at 2048/8	Bookkeeping versus recomputation.
CPU W8A8	TPS 1.27–2.49×; RSS 1.12–1.66×	Speed improvement, not resident-memory reduction.

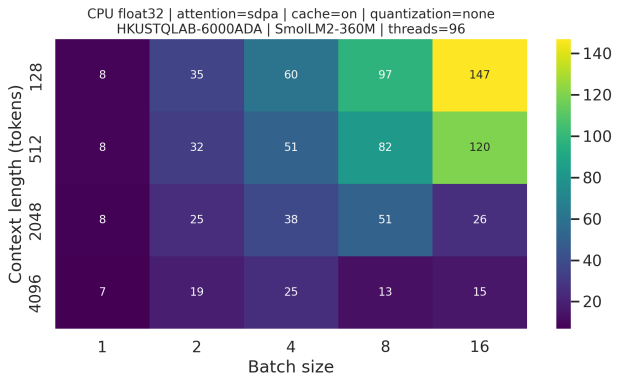
B. Attention and Cache Crossovers

Table I summarizes the intervention effects. On CPU, eager/SDPA median TTFT grows from 1.00× at 128/1 to 6.23× at 4096/4; TPOT grows from 1.11× to 4.09×. On GPU, the TTFT ratio grows from 1.40× to 9.34×, while TPOT stays near a 1.30× eager penalty. The increasing long-context advantage and the observed fused kernel support an operator-level explanation rather than a label-only comparison.

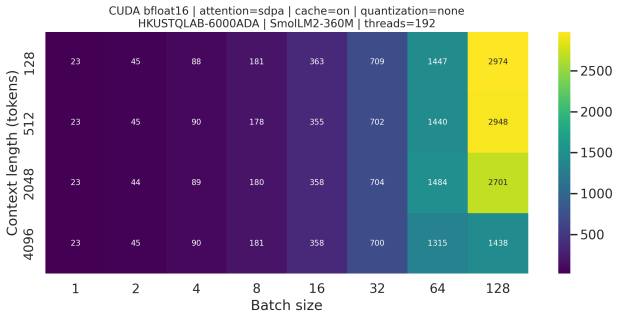
Caching shows a workload-dependent crossover. On CPU, cache-off median TPOT is 1.85–17.62× cache-on at five repeated points. The single 2048/8 boundary reaches 88.99× and takes 891.7 s; it is reported as a boundary, not a percentile. On GPU, cache-off is 1–5% faster at five smaller loads, then becomes 4.22× slower at 2048/8. For this 360M model, cache bookkeeping can exceed recomputation before the sequence–batch product becomes sufficiently large. Production systems therefore need explicit KV-memory management [7], although our synchronous harness does not implement PagedAttention or request scheduling.

C. Quantization, Memory, and Quality

CPU W8A8 increases median TPS by 1.27–2.49× across four points, but peak RSS is 1.12–1.66× the native path. The tested eager dynamic quantization path is therefore a speed optimization, not a resident-memory optimization. The quality cost is substantial: WikiText-2 perplexity rises from 11.27 to 44.16, while full ARC-Easy accuracy falls from 56.44% to 42.21% (normalized: 49.12% to 38.97%). Native HellaSwag accuracy is 42.83%, or 56.88% with standard length normalization, over 10,042 examples (approximately 0.49 percentage-point standard error).



(a) CPU FP32, batches 1–16



(b) RTX 6000 Ada BF16, batches 1–128

Fig. 1. Corrected median decode-throughput surfaces. Long context moves the CPU knee to smaller batches, whereas the GPU continues to gain throughput until the edge of most tested grids.

TABLE II
CPU W8A8 Quality

Metric	Native	W8A8	Change
WikiText-2 PPL	11.27	44.16	+32.89
ARC-Easy acc.	56.44%	42.21%	-14.23 pp
ARC-Easy acc._norm	49.12%	38.97%	-10.14 pp

The GPU TorchAO path is retained as a negative diagnostic. Profiling confirms 225 `aten::_int_mm` calls and 224 Ampere INT8 GEMMs, but also 9,025 device-to-host scalar transfers during one 128/1 prefill. Throughput is only about 0.21 TPS, so we make no full-grid GPU W8A8 speed claim. Functional INT8 execution is not sufficient evidence of an efficient end-to-end serving path.

D. Synthesis and Limitations

The resulting selection map is workload-shaped. Short, small GPU loads are overhead-sensitive and may not benefit from cache state. Larger batches increase utilization until context-driven latency and activation/KV memory tighten the frontier. Long-context prefill strongly favors SDPA. Long, batched decode favors caching. CPU W8A8 is attractive only when its speedup justifies higher RSS and the measured quality loss is acceptable, or when a better calibration and quantization recipe restores accuracy.

These are synchronous, model-only measurements. They exclude tokenization, networking, continuous batching, admission control, request queues, prefix sharing, and multi-tenant effects. The study uses one small model, one CPU platform, one GPU model, and one main framework. Future work should repeat the region map with a production scheduler, larger models, static or compiled cache paths, and alternative quantization recipes.

V. Contribution Statement

Hanxiao led the benchmark framework, reproducibility controls, CPU experiments, and data correction and validation. Haiyang Peng led GPU execution, operator/cache/quantization studies, and quality evaluation. Both authors jointly designed the experiment, interpreted results, reviewed artifacts, and prepared the report and presentation.

VI. AI-Use Acknowledgment

OpenAI Codex was used for repository planning, implementation and debugging assistance, consistency audits, and drafting and editing the presentation and report. The authors executed the experiments, reviewed generated changes, verified raw and processed results against manifests and profiler evidence, reran tests, and remain responsible for all claims. AI-generated text or code was not treated as experimental evidence.

References

[1] L. B. Allal, A. Lozhkov, E. Bakouch *et al.*, "SmolLM2: When smol goes big—data-centric training of a small language model," *arXiv preprint arXiv:2502.02737*, 2025. [Online]. Available: https://arxiv.org/abs/2502.02737

[2] PyTorch, "`torch.nn.functional.scaled_dot_product_attention`," https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html, accessed: 2026-07-23.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[4] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations*, 2017.

[5] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try ARC, the AI2 reasoning challenge," in *arXiv preprint arXiv:1803.05457*, 2018.

[6] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 4791–4800.

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.